

Design and Architecture

P2P Car Rental System

Course	MSE800 Professional Software Engineering
Assignment	Assignment 1 - Task 1
Version	1.0.0
Date	May 2026

1. Introduction

This document shows the design and architecture of the P2P Car Rental System. It contains all 8 UML diagrams used in the project. The diagrams cover structure (ERD, Class), behavior (Use Case, Sequence), and process flows (4 Activity diagrams).

1.1 Architecture Overview

The system uses a 4-layer architecture. Each layer has a clear responsibility. Layers only talk to the layer directly below them.

Layer	Folder	Purpose
1. Presentation	ui/	CLI menus for users.
2. Service	services/	Business logic (booking, auth, admin).
3. Model	models/	OOP entities (User, Car, Booking).
4. Data Access	database/	SQLite database connection.

1.2 Design Patterns

The system uses three design patterns from the Gang of Four catalog (Gamma et al., 1994):

Pattern	Used in	Purpose
Singleton	DatabaseManager	One database connection across the app.
Factory	UserFactory	Create Owner, Renter, or Admin from a role string.
Strategy	InsuranceStrategy	Different insurance plans with the same interface.

2. Entity Relationship Diagram

The ERD shows the database schema. There are 5 tables: USERS, CARS, BOOKINGS, REVIEWS, and INSURANCE_CLAIMS. The diagram shows primary keys (PK), foreign keys (FK), and the unique key (UK) on the email column.

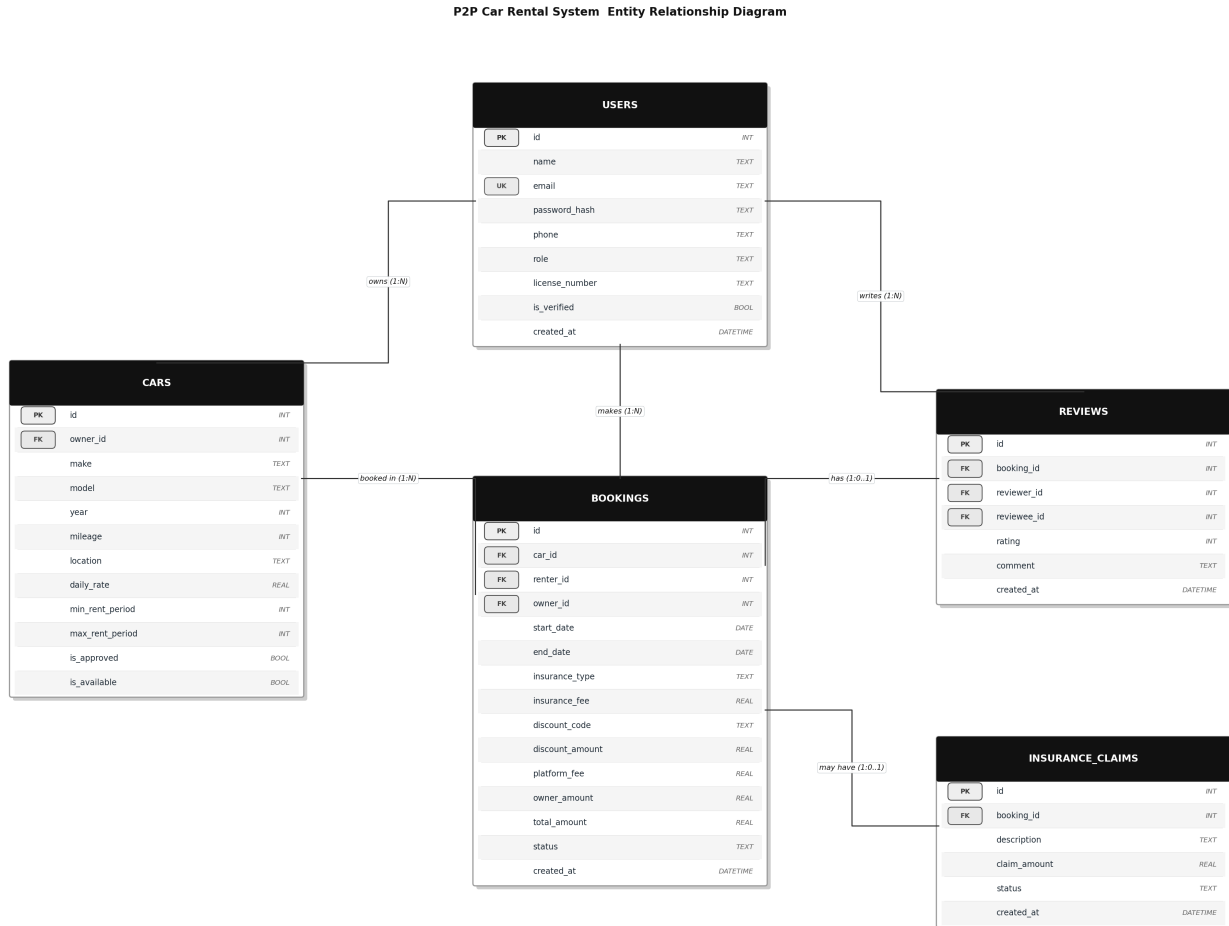


Figure 1: ERD with 5 tables, 8 foreign keys, and 1 unique key.

2.1 Key Design Decisions

- **Email is unique** in the **USERS** table. Two users cannot have the same email.
- **Role is a CHECK constraint**: only owner, renter, or admin values are allowed.
- **Bookings have 3 foreign keys**: car_id, renter_id, and owner_id. The owner_id is denormalized for fast queries.
- **Booking status is a CHECK constraint**: pending, approved, rejected, cancelled, or completed.
- **discount_code and discount_amount** are saved in the **BOOKINGS** table for audit and reporting.

3. Use Case Diagram

The Use Case Diagram shows what each actor can do. There are 3 actors (Owner, Renter, Admin) and 19 use cases. The diagram also shows "include" (mandatory) and "extend" (optional) relationships.

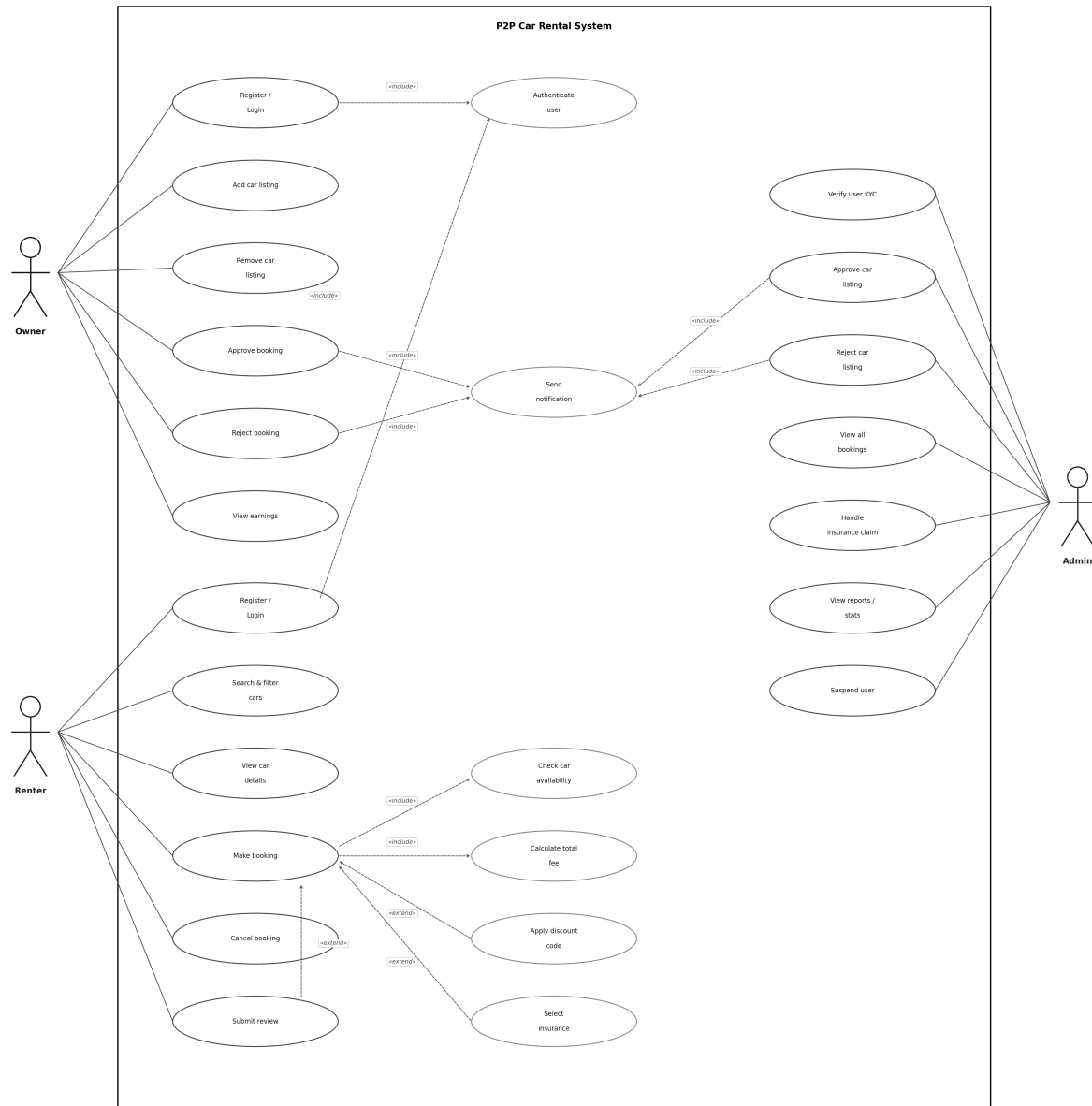


Figure 2: Use Case Diagram with 3 actors and 19 use cases.

3.1 Actor Responsibilities

Actor	Main use cases
Owner	Register, login, add and remove car listings, approve or reject bookings, view earnings.
Renter	Register, login, search cars, view details, make and cancel bookings, submit reviews.
Admin	Verify users (KYC), approve or reject car listings, handle insurance claims, view platform reports, suspend users.

3.2 Relationship Types

- **"Include"**: A mandatory subtask. For example, "Make booking" includes "Check availability".
- **"Extend"**: An optional subtask. For example, "Make booking" can extend with "Select insurance" or "Apply discount code".

4. Class Diagram

The Class Diagram shows the OOP structure. It has the abstract User base class, three subclasses (Owner, Renter, Admin), three pattern classes (DatabaseManager, UserFactory, Insurance), and entity classes (Car, Booking, Review, InsuranceClaim).

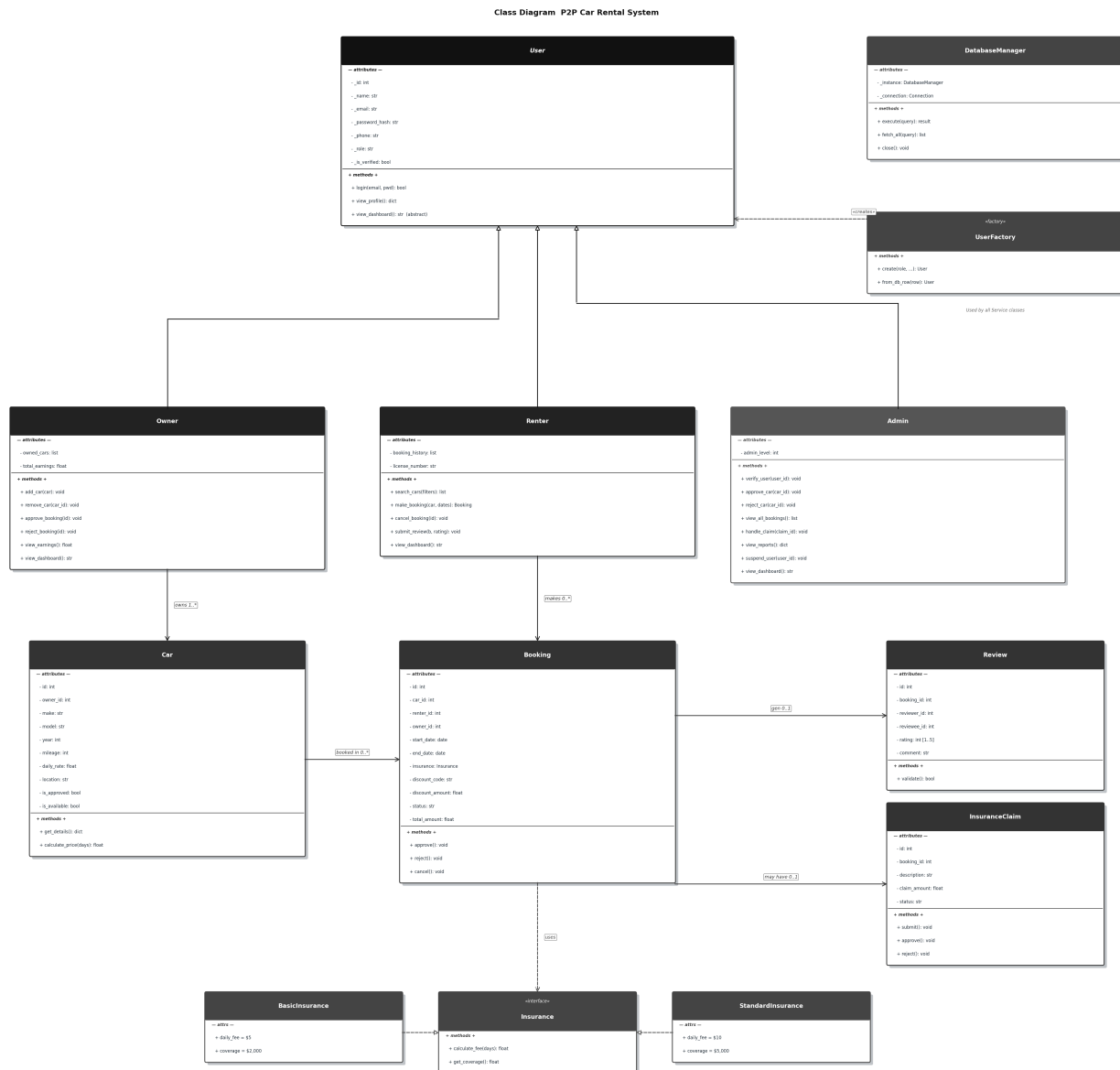


Figure 3: Class Diagram showing the OOP hierarchy.

4.1 OOP Principles in the Diagram

Principle	How it is shown
Abstraction	User class is abstract (cannot be created directly). It uses <code>abc.ABC</code> and <code>@abstractmethod</code> for <code>view_dashboard()</code> .
Encapsulation	Attributes use a single underscore prefix (<code>_id</code> , <code>_name</code>). They are accessed through <code>@property</code> methods.
Inheritance	Owner, Renter, and Admin extend User. They reuse all attributes and methods.
Polymorphism	Each subclass overrides <code>view_dashboard()</code> to show a different output for each role.

5. Sequence Diagram: Make Booking

The Sequence Diagram shows the order of messages between objects when a renter makes a booking. This is the most complex use case in the system. The diagram shows 9 lifelines (participants) and the messages between them.

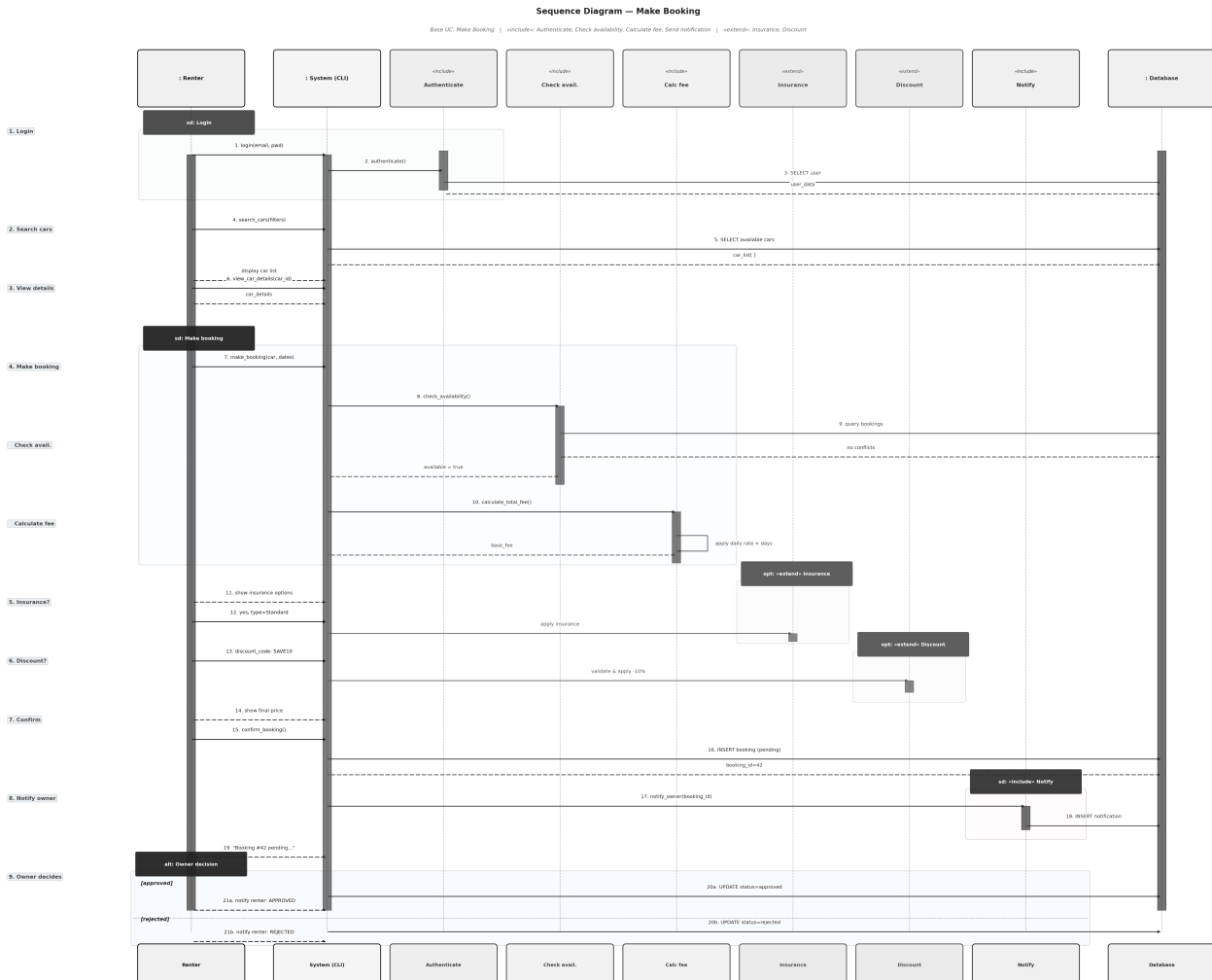


Figure 4: Sequence Diagram for the Make Booking use case.

5.1 Steps Shown in the Diagram

#	Step	Notes
1	Renter logs in.	Includes Authenticate.
2	Renter searches for cars.	SQL with filters.
3	Renter views car details.	Get one car by ID.
4	Renter makes a booking.	Main use case.
5	System checks availability.	Include relationship.
6	System calculates the fee.	Include relationship.
7	Renter selects insurance.	Extend (optional).
8	Renter applies a discount code.	Extend (optional).
9	System confirms the booking.	INSERT into database.
10	System notifies the owner.	Include relationship.

6. Activity Diagrams

Activity Diagrams show the flow of activities for a use case. They use decisions (diamonds), forks and joins (parallel steps), and start/end nodes. The system has 4 activity diagrams.

6.1 Make Booking

The full process for a renter making a booking. The fork after step 4 shows that "Check availability" and "Calculate total fee" run in parallel.

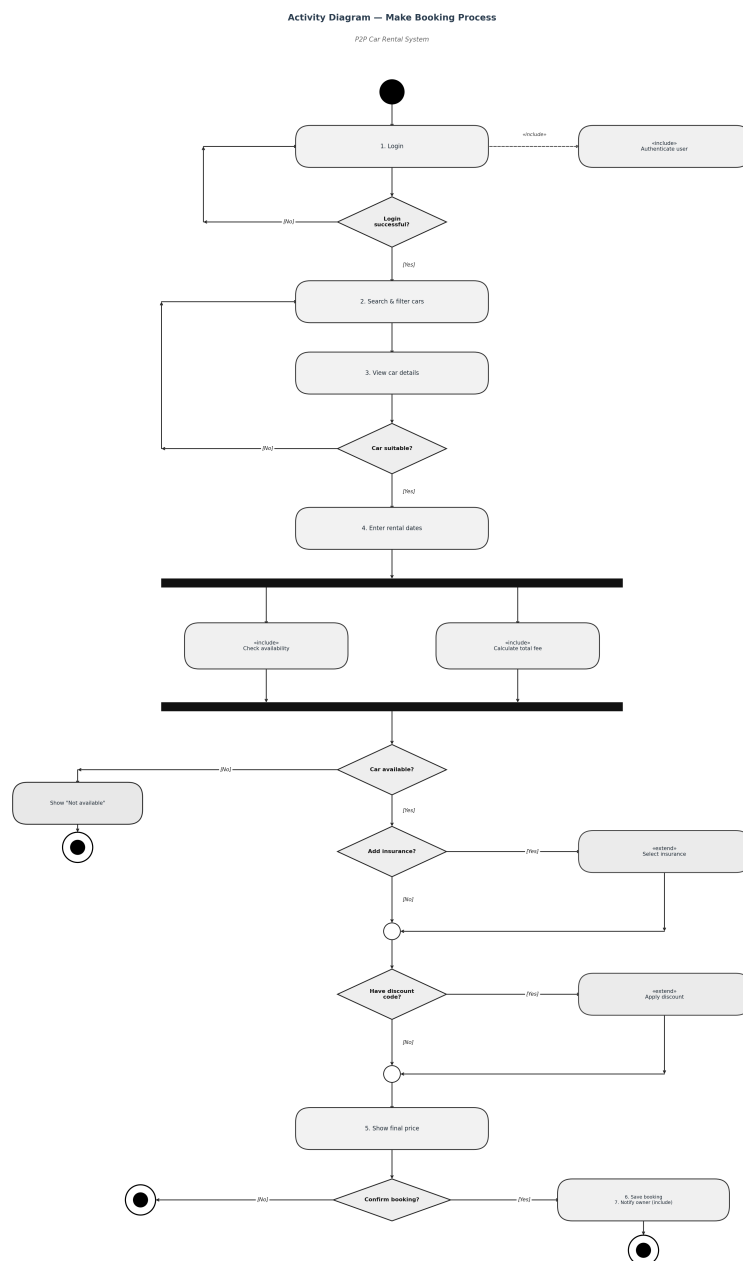


Figure 5: Activity Diagram for the Make Booking process.

6.2 Add Car Listing

This diagram has two swimlanes: Owner and Admin. The owner submits a car listing, then the admin reviews and approves or rejects it.

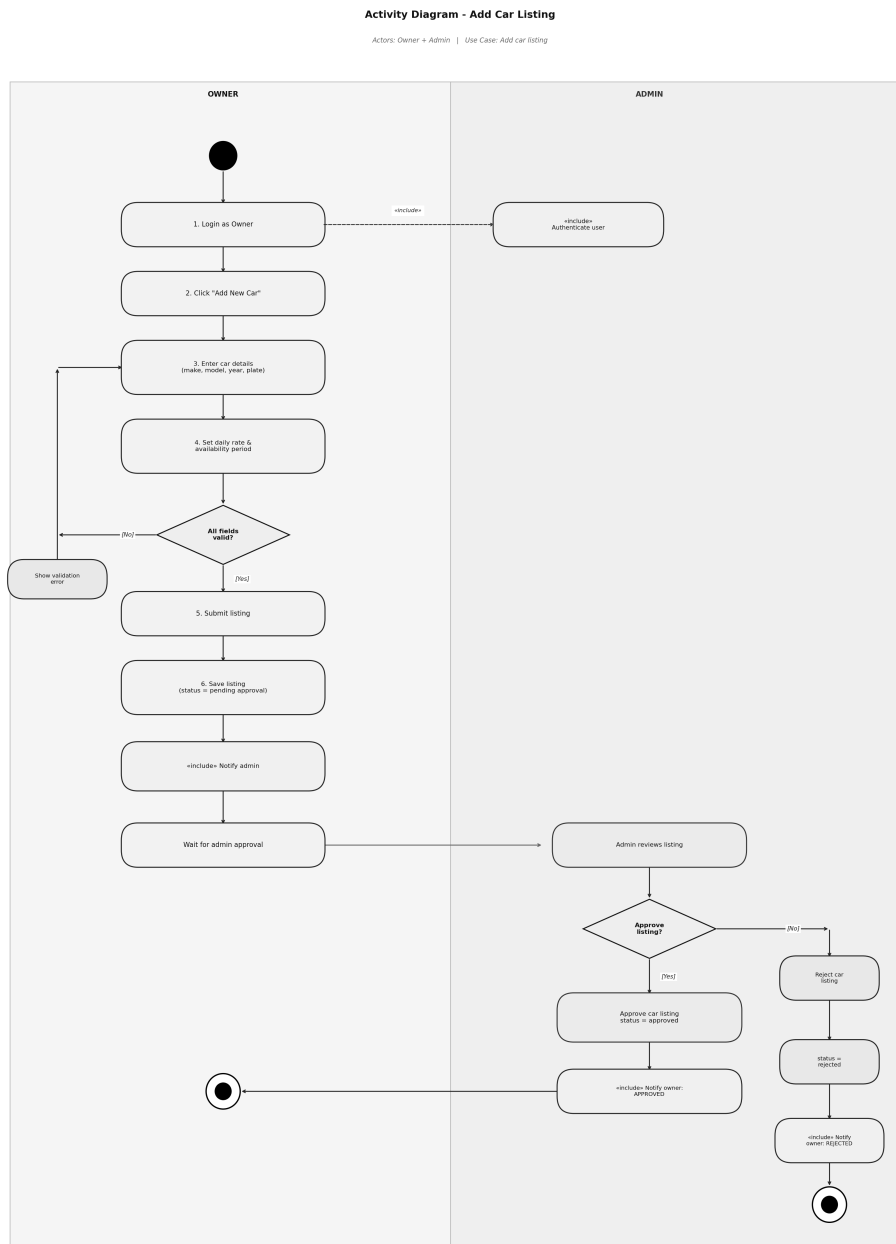


Figure 6: Activity Diagram for adding a car listing.

6.3 Approve or Reject Booking

When a renter requests a booking, the owner gets a notification. The owner reviews the booking and decides to approve or reject it. The diagram shows both paths clearly.

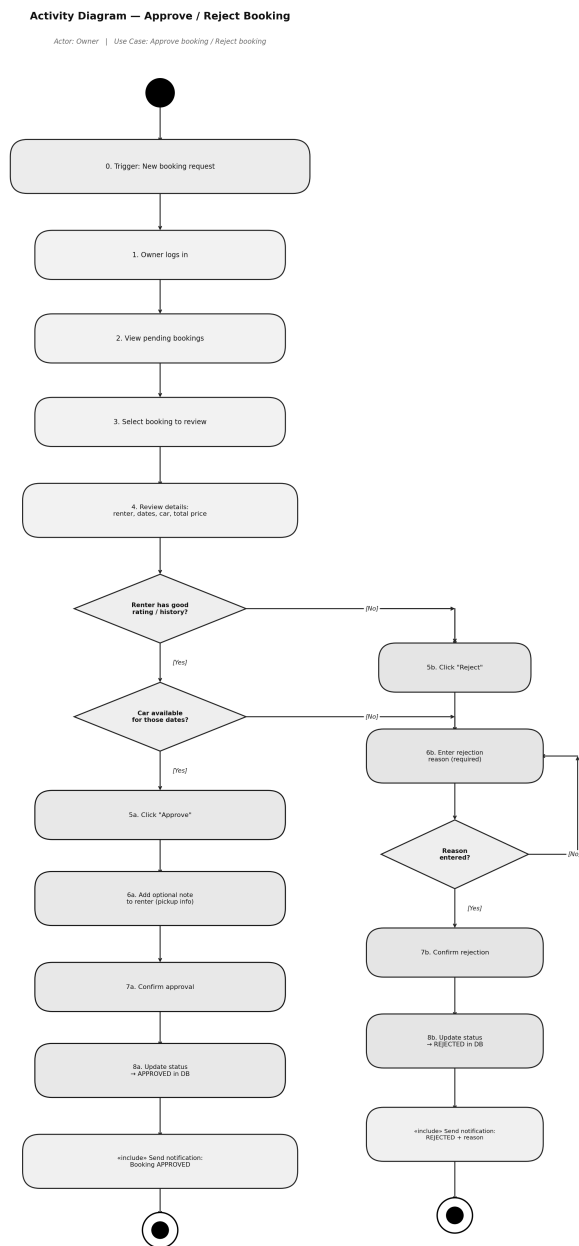


Figure 7: Activity Diagram for approving or rejecting a booking.

6.4 Handle Insurance Claim

This diagram has three swimlanes: Owner, Admin, and Renter. When a car is damaged, the owner files a claim. The renter can submit a defense statement. The admin makes the final decision (approve, partial payout, or reject).

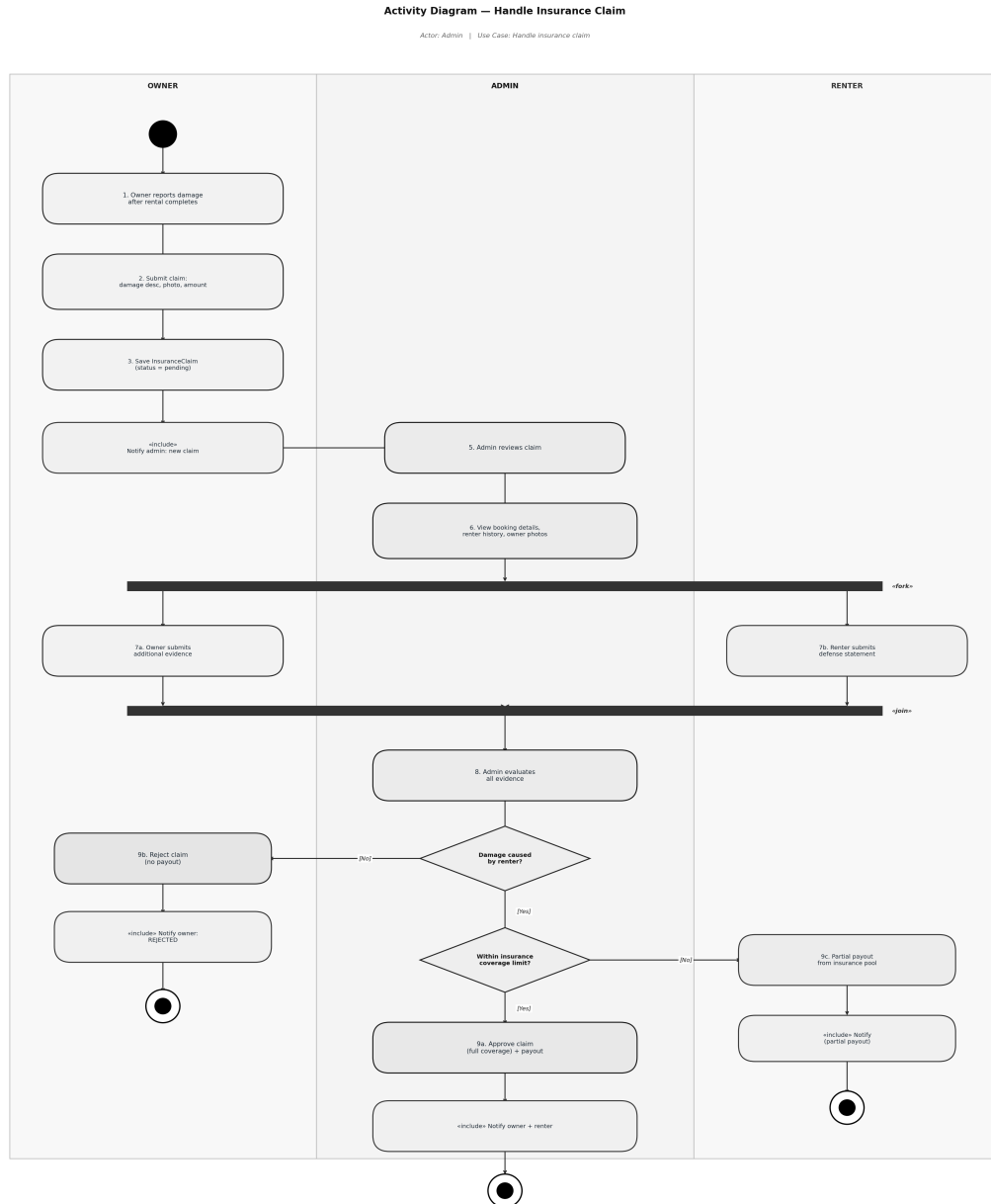


Figure 8: Activity Diagram for handling an insurance claim.

7. Conclusion

These 8 UML diagrams describe the full design of the P2P Car Rental System. The diagrams cover the structure (ERD, Class), the behavior (Use Case, Sequence), and the process flows (4 Activity diagrams). Together they show how the system works from different points of view.

The design follows standard software engineering practices. The 4-layer architecture keeps the code organized. The three design patterns solve specific problems in a clean way. The OOP principles are used where they make sense.

References

- Gamma, E., Helm, R., Johnson, R., and Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston: Addison-Wesley.
- OMG (2017). *Unified Modeling Language Specification, Version 2.5.1*. Needham: Object Management Group.